Chittagong University of Engineering & Technology



Department of Electronics and Telecommunication Engineering

PROJECT REPORT

Design and Implementation of an 8-bit SAP-1 Microprocessor with Assembler

Course No. : ETE 404
Course Title : VLSI Technology Sessional
Date of Submission :

Name : Tasir Mahtab Haque
ID : 2108008
Level : 04
Term : I
Group : 01

Contents

1 Introduction **3**

2 Project Overview and Objectives **3**
2.1 Project Goal 3
2.2 Basic Processor Specifications 3

3 System Architecture **4**
3.1 Overall Block Diagram 4
3.2 Datapath and Control Path 5

4 Hardware Modules **5**
4.1 General Purpose Registers (A and B) 6
4.2 Arithmetic Logic Unit (ALU) 6
4.3 Instruction Register (IR) 7
4.4 Program Counter (Loadable) 7
4.5 Memory Section (16x8 SRAM) and Decoder 8
4.6 Barrel Shifter and Ring Rotator 9
4.7 Conditional Flags: Carry Register & Zero Detector 9
4.8 RAM Auto-Loader 10
4.9 Ring Counter and Control Sequencer 11

5 Instruction Set Architecture (ISA) **11**
5.1 Instruction Categories 12

6 Timing, Micro-Operations, and Control Equations **12**
6.1 Common Fetch Cycle 12
6.2 Instruction Execution Tables 12
6.3 Hardwired Control Sequencer Equations 13

7 Software Component: Custom Assembler **14**
7.1 7.1 Assembler Features 14
7.2 Output Generation 14

8 Testing and Demonstration Programs **14**
8.1 Test 1: Basic Arithmetic and Data Storage 14
8.2 Test 2: Logical Operations and CLR 15
8.3 Test 3: Carry-Based Branching (JC) 15
8.4 Test 4: Variable Magnitude Shift and Rotate 16

9 Challenges and Debugging **16**

10 Unique Features **17**

11 Limitations and Future Improvements **17**

12 Conclusion **18**

List of Figures

1 Overall Block Diagram of the 8-bit SAP-1 System 5
2 8-bit Synchronous Register 6
3 Arithmetic Logic Unit (ALU) 6
4 Instruction Register Module 7
5 4-bit Loadable Program Counter 7
6 4-to-16 Decoder 8
7 16x8 SRAM Block 8
8 8-bit Barrel Shifter 9
9 8-bit Ring Rotator 9
10 RAM Auto-Loader Circuitry 10
11 Timing Ring Counter 11
12 Hardwired Control Sequencer 11

List of Tables

1 Basic Processor Specifications 4
2 Complete 16-Instruction ISA 12
3 Common Fetch Sequence 12
4 Selected Execution Sequences 13

Introduction

The objective of this project was to design and implement a fully operational, custom 8-bit SAP-1 (Simple-As-Possible) microprocessor architecture using Logisim, integrating both hardware and software components for automatic program execution. The processor builds upon the fundamental principles of computer architecture but significantly extends the traditional educational SAP-1 model to include robust computational capabilities.

The hardware section was constructed from fundamental digital modules including an 8-bit Accumulator, an 8-bit B-Register, a multi-function Arithmetic Logic Unit (ALU), a 4-to-16 opcode decoder, a 16×8 SRAM memory block, an instruction register, a program counter, a timing ring counter, and a comprehensive hardwired control sequencer. To support advanced data manipulation, dedicated 8-bit barrel-shifter and ring-rotator modules were designed and implemented, enabling variable-distance logical shifts and cyclic data rotations ranging from 0 to 7 positions in a single clock cycle.

To complement the hardware, a custom assembler was developed in Python. This assembler translates human-readable assembly source code into 8-bit machine code compatible with the SAP-1 instruction format, producing a hexadecimal image file. This machine code is seamlessly loaded into the processor's SRAM through a custom-built RAM Auto-Loader that operates via a debugging protocol, enabling program initialization without manual bit-toggling. Once memory is loaded, the microprocessor executes the program autonomously.

Project Overview and Objectives

Project Goal

The primary goal was to design and implement a fully functional, automatic 8-bit SAP-1 microprocessor system that establishes a complete end-to-end design flow—from instruction design and memory interfacing to automatic assembly code compilation and execution.

Basic Processor Specifications

To maintain a practical implementation complexity, the processor avoids overly complex operations (e.g., multiplication, division, stack operations) but implements highly versatile bit-manipulation and arithmetic capabilities. Table 1 outlines the core architectural specifications.

Feature	Specification
Processor data width	8 bits
Instruction width	8 bits
Opcode width	4 bits
Operand/address width	4 bits
Number of instructions	16
Number of memory locations	16
Memory organization	16×8 bits
Program Counter width	4 bits
Memory Address Register width	4 bits
Accumulator width	8 bits
B Register width	8 bits
Main data bus width	8 bits
Shift/rotate amount range	0–7 positions (3 bits)
Control implementation	Hardwired control sequencer
Assembler language	Python

Table 1: Basic Processor Specifications

System Architecture

Overall Block Diagram

The processor utilizes a common 8-bit data bus to route information between storage elements and functional units. The architecture ensures that only one module drives the bus at any given time to prevent bus contention.

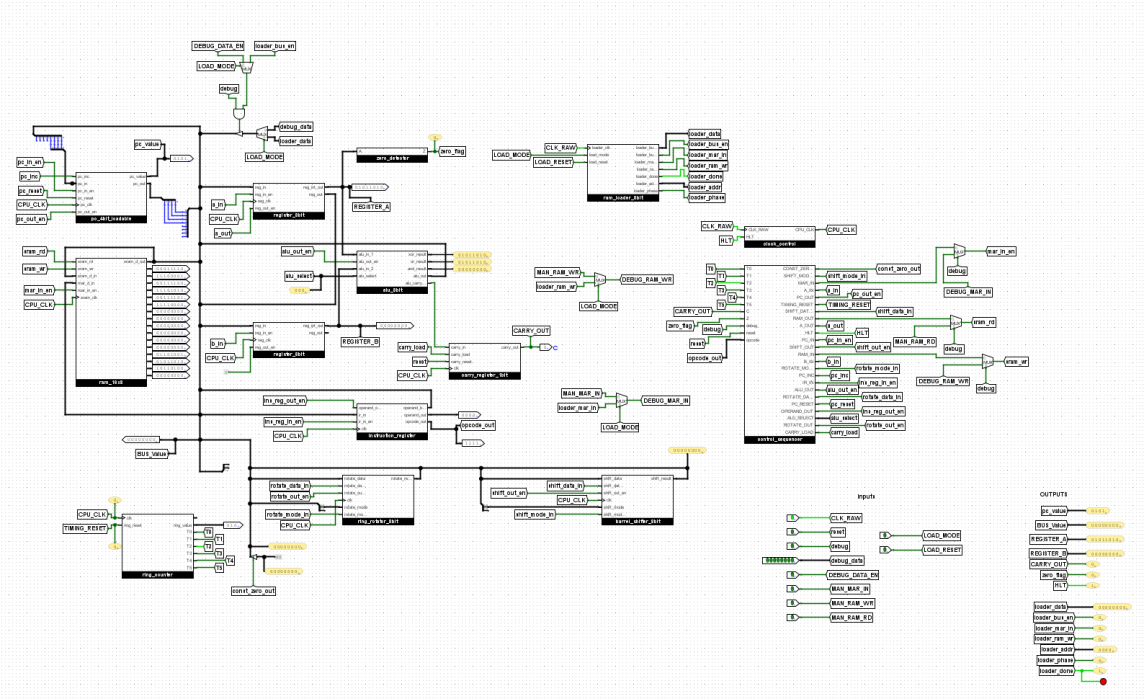


Figure 1: Overall Block Diagram of the 8-bit SAP-1 System

Datapath and Control Path

The data-processing path revolves around the Accumulator (A) and B Register. The ALU, Barrel Shifter, and Ring Rotator all take input from the Accumulator, perform their designated operations, and route the output back onto the common bus to be restored in the Accumulator or Memory. The Control Sequencer combines timing pulses (`T0`-`T5`), the decoded 4-bit Opcode, the Carry flag (`C`), and the Zero condition (`Z`) to orchestrate these data movements.

Hardware Modules

This section details the construction and working principles of the core hardware components designed in Logisim.

General Purpose Registers (A and B)

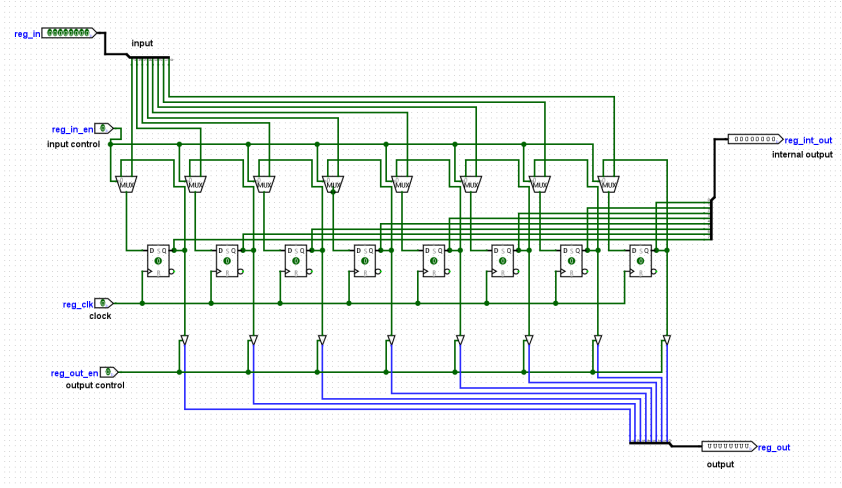


Figure 2: 8-bit Synchronous Register

General-purpose registers are used for temporary data storage. The Accumulator (A) acts as the primary fast-access storage element holding intermediate results and supplying the main operand for the ALU, Shifter, and Rotator. The B Register exclusively holds the second operand for the ALU.

Working Principle: Implemented using D-type flip-flops. When the `reg_in_en` signal is asserted, the 8-bit bus data is latched on the rising clock edge. When `reg_out_en` is activated, contents are driven onto the bus.

Arithmetic Logic Unit (ALU)

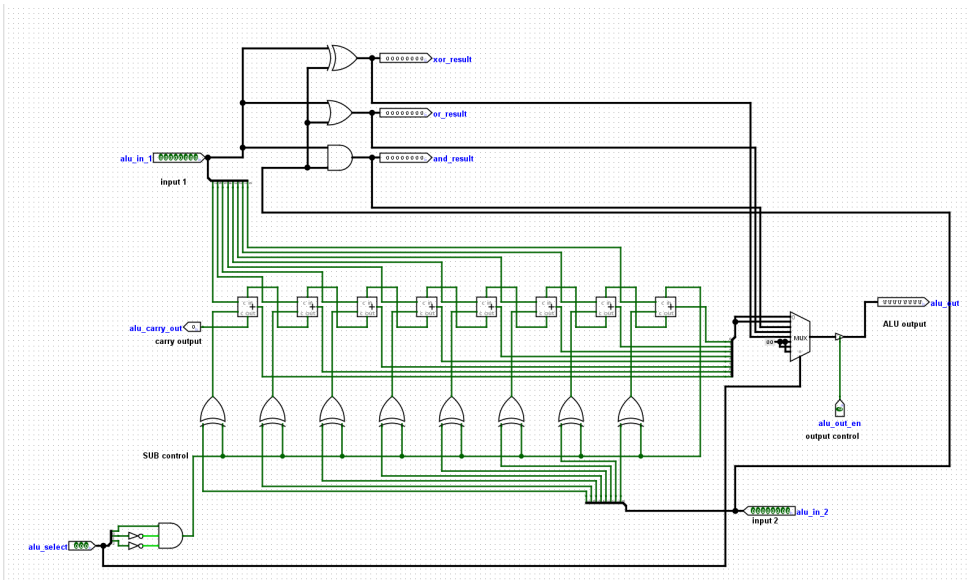


Figure 3: Arithmetic Logic Unit (ALU)

The ALU performs mathematical and logical functions. It accepts inputs from Accumulator A and Register B. It is capable of executing `ADD`, `SUB` (using 2's complement

logic), `AND`, `OR`, and `XOR`. A multiplexer selects the correct operation output based on a 3-bit `ALU_SELECT` signal generated by the sequencer. It also outputs a Carry-out bit for arithmetic operations.

Instruction Register (IR)

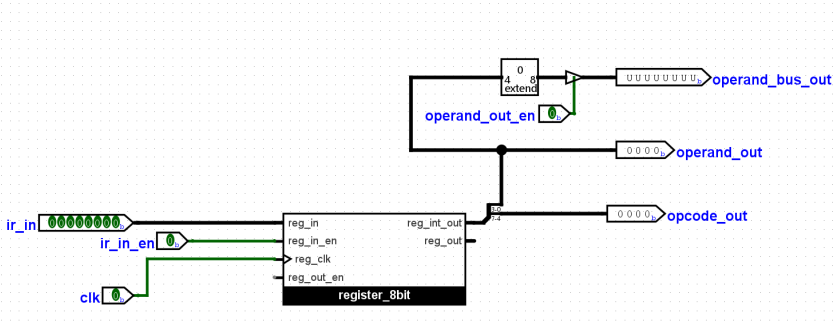


Figure 4: Instruction Register Module

The IR holds the currently executing 8-bit instruction fetched from SRAM. The register continuously routes its top 4 bits (Opcode) to the Control Sequencer via a 4-to-16 decoder. Its lower 4 bits (Operand) can be driven onto the bus via `IR_OPERAND_OUT` for memory addressing, or fed directly into the Shifter/Rotator magnitude controls.

Program Counter (Loadable)

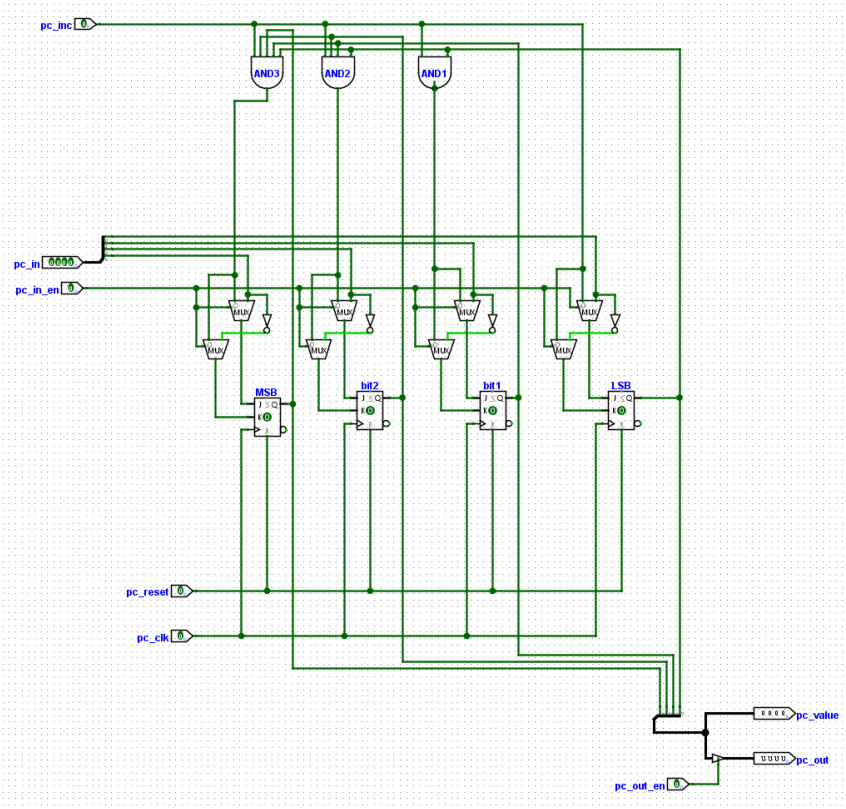


Figure 5: 4-bit Loadable Program Counter

The Program Counter (PC) stores the 4-bit address of the next instruction. Implemented as a synchronous counter, it increments sequentially via `PC_INC`. For conditional and unconditional jumps (`JMP`, `JC`, `JNZ`), it acts as a loadable register, importing a new target address from the bus when `PC_IN` is asserted.

Memory Section (16x8 SRAM) and Decoder

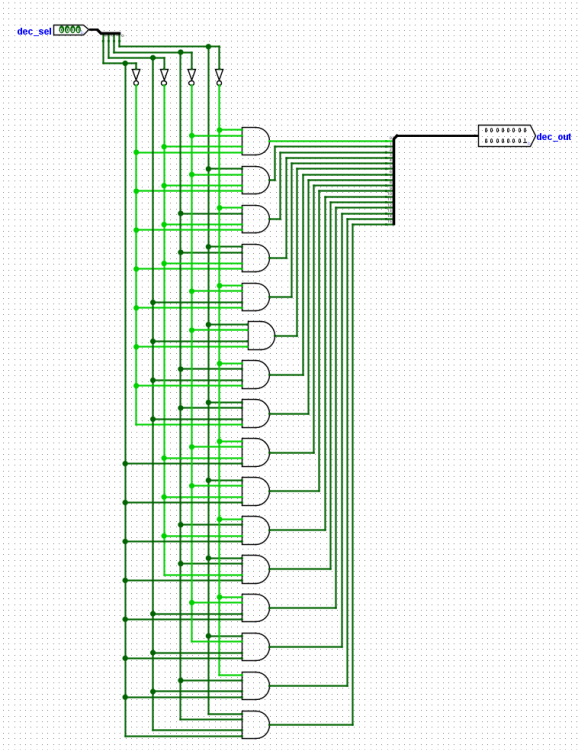


Figure 6: 4-to-16 Decoder

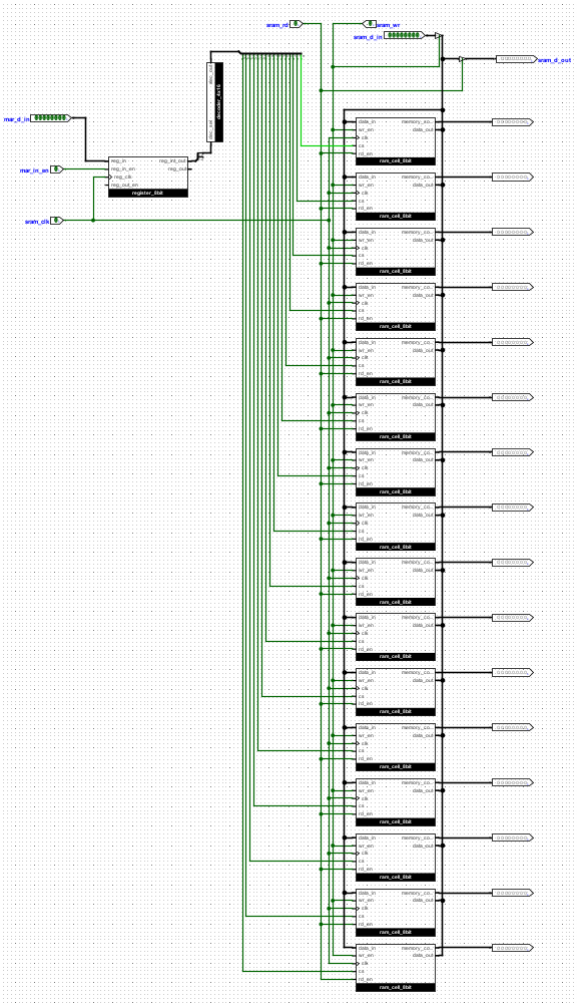


Figure 7: 16x8 SRAM Block

The SRAM block serves as the primary memory, organized into 16 words of 8 bits. The 4-bit address input goes into the Memory Address Register (MAR), which selects one of the 16 memory cells using the 4-to-16 Decoder. `RAM_RD` and `RAM_WR` signals control data flow to and from the common bus.

Barrel Shifter and Ring Rotator

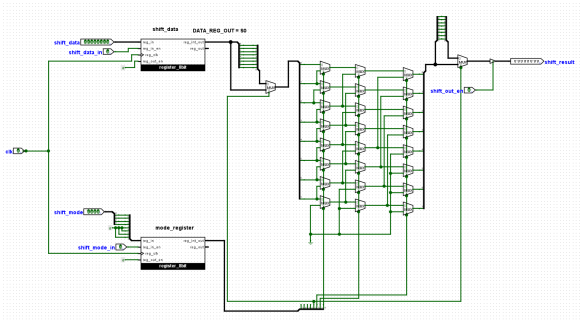


Figure 8: 8-bit Barrel Shifter

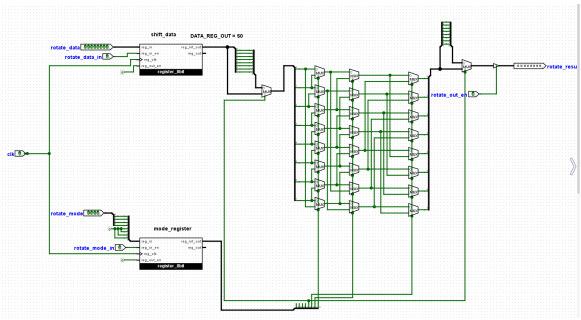


Figure 9: 8-bit Ring Rotator

To bypass the limitations of single-bit shift operations, dedicated variable-distance combinational modules were built using cascaded multiplexer stages.

- The **Barrel Shifter** performs logical shifting (filling with zeros).
- The **Ring Rotator** performs cyclic rotation (wrapping bits to the opposite side).

Both utilize the operand's 3 LSBs to determine the operation magnitude (0 to 7 positions) and the 4th bit (MSB) to determine direction (0 for Left, 1 for Right).

Conditional Flags: Carry Register & Zero Detector

Carry Register: A 1-bit synchronous D-flip-flop that latches the carry-out from the ALU during `ADD` and `SUB` instructions. This flag is evaluated during the `JC` (Jump if Carry) instruction.

Zero Detector: An 8-input NOR logic gate continuously monitors the Accumulator bus. If all 8 bits are 0, the `Z` flag becomes 1. This is evaluated during the `JNZ` (Jump if Not Zero) instruction.

RAM Auto-Loader

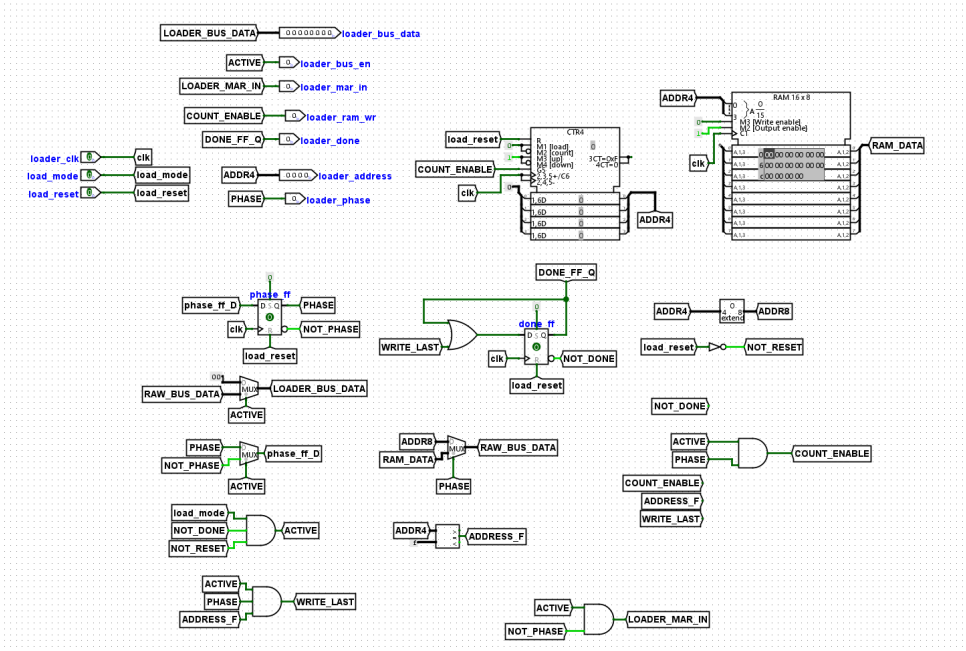


Figure 10: RAM Auto-Loader Circuitry

The RAM Auto-Loader is a custom state-machine module designed to facilitate the automatic loading of hex machine code into the SRAM. When `LOAD_MODE` is active, it generates 32 sequential clock pulses (2 pulses per address: Address Phase and Write Phase) to clone 16 bytes of data from a built-in ROM component directly into the processor's active RAM, removing the need for manual bit-toggling.

Ring Counter and Control Sequencer

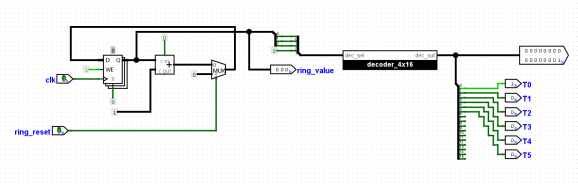


Figure 11: Timing Ring Counter

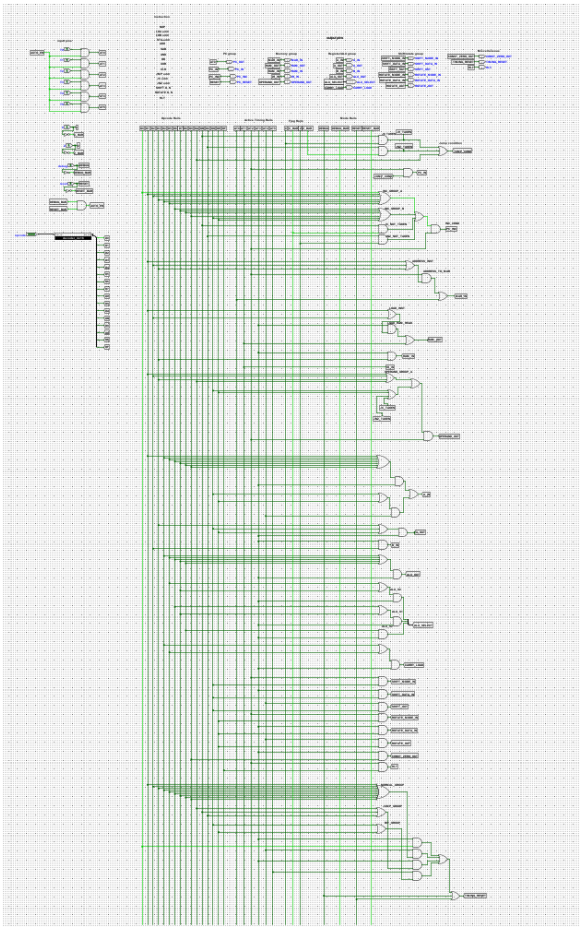


Figure 12: Hardwired Control Sequencer

The **Ring Counter** generates six distinct sequential timing pulses (`T0` to `T5`).
The **Control Sequencer** is the "brain" of the processor. It takes the timing state, the decoded opcode ($D_0 - D_{15}$), and flag states as inputs to massive combinational logic arrays (AND/OR gates). It asserts the precise micro-operation control signals (e.g., `A_IN`, `PC_INC`) at the exact correct clock cycle to execute instructions.

Instruction Set Architecture (ISA)

The processor features a custom 8-bit Instruction Set Architecture. The upper 4 bits define the opcode, allowing exactly 16 instructions, ensuring no wasted decode space. The lower 4 bits serve as the operand (memory address or shift/rotate parameters).

Hex	Opcode	Mnemonic	Operand	Register-Transfer Operation
0	0000	`NOP`	Ignored	No operation
1	0001	`LDA addr`	4-bit Address	$A \leftarrow RAM[addr]$
2	0010	`LDB addr`	4-bit Address	$B \leftarrow RAM[addr]$
3	0011	`STA addr`	4-bit Address	$RAM[addr] \leftarrow A$
4	0100	`ADD`	Ignored	$A \leftarrow A + B$, $C \leftarrow Carry$
5	0101	`SUB`	Ignored	$A \leftarrow A - B$, $C \leftarrow Borrow$
6	0110	`AND`	Ignored	$A \leftarrow A$ AND B
7	0111	`OR`	Ignored	$A \leftarrow A$ OR B
8	1000	`XOR`	Ignored	$A \leftarrow A$ XOR B
9	1001	`CLR`	Ignored	$A \leftarrow 00000000_b$
A	1010	`JMP addr`	Target Address	$PC \leftarrow addr$
B	1011	`JC addr`	Target Address	If $C = 1$, $PC \leftarrow addr$
C	1100	`JNZ addr`	Target Address	If $A \neq 0$, $PC \leftarrow addr$
D	1101	`SHIFT D, N`	$D\ N_2\ N_1\ N_0$	Logically shift A by N bits ($D = 0$: L, $D = 1$: R)
E	1110	`ROTATE D, N`	$D\ N_2\ N_1\ N_0$	Cyclically rotate A by N bits ($D = 0$: L, $D = 1$: R)
F	1111	`HLT`	Ignored	Halt processor execution

Table 2: Complete 16-Instruction ISA

Instruction Categories

- **Address Instructions (`LDA, LDB, STA, JMP, JC, JNZ`):** Require a 4-bit address (0-15).
- **No-Operand Instructions (`NOP, ADD, SUB, AND, OR, XOR, CLR, HLT`):** Operand bits are ignored (defaulted to 0000 by assembler).
- **Packed Operand Instructions (`SHIFT, ROTATE`):** The 4-bit operand is packed as `D N2 N1 N0`. The MSB specifies direction, while the 3 LSBs specify magnitude.

Timing, Micro-Operations, and Control Equations

The execution of each instruction is governed by the 6-phase timing cycle (`T0-T5`).

Common Fetch Cycle

Every instruction undergoes the identical fetch sequence during the first three clock cycles:

Time	Control Signals Asserted	Micro-Operation
`T0`	`PC_OUT`, `MAR_IN`	$MAR \leftarrow PC$
`T1`	`RAM_OUT`, `IR_IN`	$IR \leftarrow RAM[MAR]$
`T2`	`PC_INC`	$PC \leftarrow PC + 1$

Table 3: Common Fetch Sequence

Instruction Execution Tables

Execution is instruction-specific and begins at `T3`.

Instruction	Time	Control Signals Asserted
NOP	T3	TIMING_RESET
LDA	T3	IR_OPERAND_OUT, MAR_IN
	T4	RAM_OUT, A_IN
	T5	TIMING_RESET
STA	T3	IR_OPERAND_OUT, MAR_IN
	T4	A_OUT, RAM_IN
	T5	TIMING_RESET
ADD	T3	ALU_OUT, A_IN, CARRY_LOAD, ALU_SELECT=000
	T4	TIMING_RESET
JMP	T3	IR_OPERAND_OUT, PC_IN
	T4	TIMING_RESET
JC	T3	If $C = 1$: IR_OPERAND_OUT, PC_IN
	T4	TIMING_RESET
SHIFT	T2	IR_OPERAND_OUT, SHIFT_MODE_IN
	T3	A_OUT, SHIFT_DATA_IN
	T4	SHIFT_OUT, A_IN
	T5	TIMING_RESET
HLT	T3	HLT

Table 4: Selected Execution Sequences

Hardwired Control Sequencer Equations

To implement the execution table in Logisim, Boolean equations were derived for every control pin. Below are key examples of these equations (where D_n represents the decoded opcode, and AT_n represents the active timing state):

Helper Signals:

$$AUTO_EN = \overline{debug} \cdot \overline{reset}$$
$$AT_n = AUTO_EN \cdot T_n \quad (\text{for } n = 0 \text{ to } 5)$$

Program Counter Controls:

$$PC_OUT = AT_0$$
$$PC_INC = AT_2 \cdot (D_0 + D_1 + D_2 + D_3 + D_4 + D_5 + D_6 + D_7 + D_8 + D_9 + D_B\overline{C} + D_C Z + D_D + D_E)$$
$$PC_IN = AT_3 \cdot (D_A + D_B \cdot C + D_C \cdot \overline{Z})$$

RAM and Registers:

$$MAR_IN = AT_0 + AT_3 \cdot (D_1 + D_2 + D_3)$$
$$RAM_OUT = AT_1 + AT_4 \cdot (D_1 + D_2)$$
$$A_IN = AT_4 \cdot (D_1 + D_4 + D_5 + D_6 + D_7 + D_8 + D_9) + AT_4 \cdot (D_D + D_E)$$

ALU Controls:

$$ALU_OUT = AT_3 \cdot (D_4 + D_5 + D_6 + D_7 + D_8)$$
$$CARRY_LOAD = AT_3 \cdot (D_4 + D_5)$$

Automatic Timing Reset:

$$\begin{aligned} TIMING_RESET = {} & AT_3 \cdot D_0 \\ & + AT_5 \cdot (D_1 + D_2 + D_3 + D_D + D_E) \\ & + AT_4 \cdot (D_4 + D_5 + D_6 + D_7 + D_8 + D_9 + D_A + D_B + D_C) \end{aligned}$$

Software Component: Custom Assembler

To bridge the gap between human programming and hardware logic, an Assembler was developed in Python (`assembler_8bit.py`).

7.1 Assembler Features

- **Tokenization and Syntax Parsing:** Separates mnemonics, operands, and ignores comments (`;`).
- **Suffix Validation:** Strictly enforces numeric suffixes to prevent base ambiguity: `b` (binary), `d` (decimal), or `h` (hexadecimal). E.g., `15d` or `0Fh`.
- **Label Resolution:** Processes jump targets dynamically via a two-pass algorithm.
- **Directives:**
  - `.ORG` modifies the addressing pointer to place data in specific RAM locations.
  - `DB` (Define Byte) initializes an 8-bit static variable in memory.

Output Generation

The assembler validates ranges (e.g., verifying address ≤ 15, shifts ≤ 7) and merges the Opcode and Operand into an 8-bit Hex string format required by the Logisim RAM component (e.g., `v3.0 hex words addressed`).

Testing and Demonstration Programs

To verify the architecture, various assembly programs were developed and executed. The following tests cover data movement, logic, conditional branching, and complex bit manipulation.

Test 1: Basic Arithmetic and Data Storage

This test verifies memory read/write, Accumulator/B-register loading, and basic addition.

Listing 1: Test 1: Arithmetic

```
START:
    LDA DATA_A      ; A <- RAM[E] = 0Ch
    LDB DATA_B      ; B <- RAM[F] = 05h
    ADD             ; A <- A + B = 11h
    STA RESULT      ; RAM[D] <- A
```

```
    HLT
.ORG 0Dh
RESULT: DB 00h
DATA_A: DB 0Ch
DATA_B: DB 05h
```

Expected Execution Result: $A = 11h$, $B = 05h$, $RAM[D] = 11h$.
Generated Hex Image: `1E 2F 40 3D F0 00 00 00 00 00 00 00 00 00 0C 05`

Test 2: Logical Operations and CLR

This verifies the ALU's bitwise logic multiplexing and the `CLR` instruction.

Listing 2: Test 2: Logic

```
START:
    LDA DATA_A      ; A = F0
    LDB DATA_B      ; B = 0F
    AND             ; F0 AND 0F = 00
    OR              ; 00 OR 0F = 0F
    XOR             ; 0F XOR 0F = 00
    LDA DATA_A
    SUB             ; F0 - 0F = E1
    CLR             ; A = 00
    HLT
.ORG 0Eh
DATA_A: DB 0F0h
DATA_B: DB 0Fh
```

Generated Hex Image: `1E 2F 60 70 80 1E 50 90 F0 00 00 00 00 00 F0 0F`

Test 3: Carry-Based Branching (JC)

This test verifies the integration of the Carry Register with the Program Counter load logic.

Listing 3: Test 4: JC Taken

```
START:
    LDA DATA_FF
    LDB DATA_01
    ADD             ; FF + 01 = 00, Carry = 1
    JC TAKEN        ; Jump should execute
    LDA BAD
    HLT
TAKEN:
    LDA GOOD
    HLT
.ORG 0Ch
GOOD:    DB 0AAh
BAD:     DB 55h
DATA_FF: DB 0FFh
DATA_01: DB 01h
```

Expected Execution Result: Because `FF + 01` yields a carry, the PC branches to `TAKEN`, skipping `LDA BAD`. Final Accumulator state is `AAh`.

Test 4: Variable Magnitude Shift and Rotate

This test validates the custom Barrel Shifter and Ring Rotator hardware, proving the processor can shift multiple bits natively in one instruction.

Listing 4: Test 8: Rotate Left and Right

```
; B4 = 10110100
; Rotate left 1  -> 01101001 = 69
; Rotate right 2 -> 01011010 = 5A
START:
    LDA DATA
    ROTATE LEFT 1d
    STA RESULT_LEFT
    ROTATE RIGHT 2d
    STA RESULT_RIGHT
    HLT
.ORG 0Ch
RESULT_LEFT:  DB 00h
RESULT_RIGHT: DB 00h
DATA:         DB 0B4h
```

Expected Execution Result: `RAM[0Ch] = 69h`, `RAM[0Dh] = 5Ah`. The hardware correctly interprets the MSB of the operand for direction and the LSBs for the magnitude loop.

Challenges and Debugging

Throughout the design, simulation, and integration phases of this 8-bit SAP-1 microprocessor, several practical engineering challenges were encountered and systematically resolved. Rather than theoretical errors, most difficulties arose during the hardware integration and automation stages:

1. **Subcircuit Integration Issues:** While individual modules (such as the ALU, registers, and decoders) passed isolated test benches successfully, initial interconnection on the main top-level canvas (`sap1_final_top`) occasionally failed to yield the expected outputs. This was primarily due to bus conflicts and improper control signal sequencing across modules. These issues were resolved by strictly verifying that only one device drove the 8-bit data bus during any given clock phase and validating the control sequencer's Boolean equations.

2. **RAM Auto-Loader Troubleshooting:** Implementing the automated memory loading mechanism presented significant hurdles. Initially, the auto-loader failed to transfer the compiled machine code from the built-in source ROM to the processor's custom SRAM due to incorrect handshaking between the `LOAD_MODE`, `LOAD_RESET`, and clock signals. By carefully calibrating the 32-pulse debug protocol and ensuring proper timing synchronization between the address and write phases, seamless program initialization was finally achieved.

3. **Rotate Output Stability during Loading:** During testing of the ring rotator module, fluctuating or unstable data appeared on the bus during transitional loading states. This was caused by the combinational rotator evaluating changing data

while the system bus was still active. The problem was successfully mitigated by incorporating isolated intermediate registers within the rotator and shifter modules, guaranteeing that computed results remained completely stable for at least one full clock cycle prior to bus transfer.

Unique Features

In addition to the standard SAP-1 architecture, several key design enhancements were implemented:

1. **100% Opcode Utilization:** All 16 potential 4-bit opcodes are functionally implemented, leaving no wasted instruction space.
2. **Conditional Execution:** The addition of a Carry flag and Zero detector transforms a linear processor into a Turing-complete control machine capable of looping (`JNZ`) and overflow handling (`JC`).
3. **Advanced Hardware Bit Manipulation:** Unlike basic processors that rely on software loops for shifting, this processor uses deep multiplexer logic to execute variable (0-7 position) shifts and rotations natively in a single clock cycle.
4. **Automated Software Toolchain:** The integration of a strict Python assembler and a RAM Auto-Loader completely modernizes the testing workflow, eliminating the tedious and error-prone process of manually typing binary into RAM cells.

Limitations and Future Improvements

Current Limitations:

- Memory is limited to 16 bytes due to the 4-bit address bus constraints.
- No Stack Pointer, meaning subroutine `CALL` and `RET` are not supported.
- No `LDI` (Load Immediate) instruction; constants must be stored in memory using `DB`.

Future Improvements:

- Expanding the instruction width or bus width to allow a larger memory addressing space (e.g., 6-bit or 8-bit addressing).
- Introducing input/output (I/O) mapped ports to interact with external peripherals.
- Adding a Stack architecture and `PUSH/POP` instructions.

Conclusion

This project successfully demonstrates the complete design and implementation of an enhanced 8-bit SAP-1 microprocessor system, integrating custom hardware modules with a dedicated software assembler.

Through modular circuit design in Logisim, the processor achieves seamless program compilation, automated memory initialization, and autonomous execution. Key architectural improvements—such as the transition to synchronous loadable registers, the integration of complex multi-bit shift/rotate combinational logic, and a conditional branching system—extend the capabilities of the classical SAP-1 significantly. The challenges addressed during development, such as bus contention and hardware timing synchronization, provided a profoundly practical understanding of digital logic, CPU datapath orchestration, and embedded systems architecture.